

BloodLink — Project Explanation

Smart Blood & Emergency Donor Network · System X 1.0 Problem-Solving Hackathon

Overview

BloodLink connects people who urgently need blood with compatible, eligible donors nearby. A request is verified, compatible donors are ranked by an explainable match score, the best donors are notified in waves (expanding the radius only when needed), donor responses are tracked, donations are confirmed, and every step feeds a dashboard with AI insights and demand forecasting. **Stack:** FastAPI (Python) backend, SQLite via SQLAlchemy, React (Vite) frontend, JWT auth. All AI runs offline with no paid API.

Complete workflow

Person needs blood → Request created (AI priority + summary + duplicate check) → Request verified by coordinator (or critical fast-track) → Compatible donor groups found → Donors filtered by eligibility & distance → Donors ranked → Top 5 within 5 km notified → Accept / decline → wave 2/3/4 if needed (10/25/50 km) → extra notifications auto-stopped when enough accept → Donation confirmed → Request fulfilled/completed → Statistics updated.

Important folders

Folder	Purpose
backend/	FastAPI server: APIs, business logic, database, AI.
backend/routers/	API endpoints grouped by area (auth, requests, donors, admin/analytics).
backend/services/	Core logic: compatibility rules, smart matcher, request manager (lifecycle + escalation), notifications, AI, locations.
backend/tests/	Unit tests and an end-to-end test of the full request workflow.
frontend/src/pages/	One screen per feature: login, new request, request list, request detail, donor home/profile, dashboard, admin screens.
frontend/src/components/	Shared UI: badges, status flow, score bars, charts, icons, toast.
frontend/dist/	Production build of the React app, served by FastAPI.

Important files

File	Purpose
backend/main.py	Starts the backend, creates tables, loads demo data on first run, runs the escalation/expiry scheduler, serves the frontend.
backend/database.py	Handles the database connection (SQLite by default, any SQL database via DATABASE_URL).
backend/models.py	Database models: User, Donor, Hospital, BloodRequest, DonorResponse, RequestEvent (timeline), Notification.
backend/schemas.py	Validates all user input (blood group, units, phone, dates, lengths).
backend/auth.py	Password hashing (PBKDF2), JWT tokens, role-based access control.
backend/serializers.py	Builds JSON responses and enforces privacy (contacts only after a donor accepts).
backend/seed.py	Demo hospitals, 81 donors and 4 weeks of history for analytics/forecasting.
services/compatibility.py	Real ABO/Rh compatibility table and donor eligibility (age, 90-day gap, availability, temporary unavailability).

File	Purpose
services/matching.py	Smart Donor Matcher: 0–100 match score from distance, response history, compatibility, rest period and response speed.
services/request_manager.py	Blood Request Manager: statuses, notification waves, radius expansion, auto-stop, donation confirmation, expiry.
services/notifications.py	Notification Service: in-app + email/SMS/push (simulated, real SMTP optional); failures are recorded, never crash the flow.
services/ai.py	AI: priority classification, summarization, duplicate detection, misuse signals, response prediction, demand forecasting.
services/locations.py	Area coordinates and distance (haversine) — no maps API needed and no home addresses stored.
frontend/src/App.jsx	App layout, role-based navigation, notifications drawer, auto-refresh.
frontend/src/api.js	Calls the backend with the login token and shows readable errors.

How the main parts connect

Layer	Connects to
React app (frontend/)	Calls /api/* endpoints with a JWT token. In development Vite forwards /api to port 8000; in production FastAPI serves the app.
Backend API (routers/)	Checks the user's role, validates input (schemas.py), calls the services, saves via models.py, returns JSON via serializers.py.
Blood Request Manager	Uses the Smart Donor Matcher to pick donors, the Notification Service to alert them, and logs every step to the request timeline.
Smart Donor Matcher	Uses compatibility.py for groups & eligibility, locations.py for distance and ai.py for response likelihood.
Database (SQLite)	Stores users, donors, hospitals, requests, responses, events and notifications.
Scheduler (main.py)	Every 60 s: escalates unanswered waves and expires overdue requests.
Dashboard	admin_routes.py aggregates statistics; ai.forecast_demand predicts next-week demand and shortage risk.

Smart matching score

Factor	Normal / Urgent / Critical weight
Distance from hospital	30 / 40 / 45 — closer is better (falls to 0 at 30 km)
Previous response behaviour	30 / 25 / 20 — Bayesian acceptance rate (new donors start at 50%)
Blood compatibility	15 — exact group best; O- kept for patients with no other option
Rest since last donation	15 / 10 / 10 — well-rested donors preferred
Response speed	10 — donors who usually reply quickly
Eligibility & availability	Hard filter — ineligible donors are shown but score 0 and are never notified

AI features

Feature	How it works
Priority classification	Keywords (accident, bleeding, ICU, surgery, dengue..., Roman Urdu terms), hours remaining and units → Normal/Urgent/Critical with confidence and reasons. AI may upgrade but never downgrade the requester's urgency.
Request summarization	Turns long text into: 'URGENT: B+ blood · 2 units · Civil Hospital · needed within 6h · for surgery'.
Duplicate detection	Fuzzy matching of patient name and description for the same hospital + blood group within 48 h.
Misuse signals	Too many requests, unverified phone, payment links, huge unit counts → flagged for admin review.
Response prediction	Estimates each donor's chance of accepting; used in ranking.
Demand forecasting	Exponentially-weighted daily demand per blood group → next-7-day prediction, trend and shortage risk vs eligible supply.

Security, verification & privacy

Hashed passwords, JWT sessions, role checks on every admin/coordinator action, input validation, phone OTP verification, coordinator verification before donors are contacted, request expiry, suspicious-request reporting, duplicate detection, admin blocking. Donors store only an area (never an exact address); names and phone numbers are revealed only after a donor accepts. Secrets come from environment variables. Final medical eligibility must be confirmed by qualified healthcare staff or the receiving blood facility.

How to run

`cd backend && pip install -r requirements.txt && uvicorn main:app` → open
`http://localhost:8000` (demo accounts: requester@demo.com, donor@demo.com, coordinator@demo.com,
admin@demo.com — password demo123).